

Especialista en enfermedades emocionales

Un agente RAG con memoria persistente y guardarraíles médicos deterministas, desplegado en Google Cloud Run.

2026-09-07 · EN PRODUCCIÓN · 37 RECURSOS GCP · 260 TESTS · \$0.12/MES

01 · RESUMEN EJECUTIVO

Qué se construyó

Un agente conversacional RAG sobre un diccionario de **1.265 términos** de enfermedades emocionales, que interpreta síntomas de forma simbólica, **cita siempre sus fuentes**, recuerda al usuario entre sesiones y **deriva a atención médica** ante señales de riesgo. Un único contenedor con backend FastAPI y frontend propio, desplegado en **Cloud Run** escalando a cero.



Pipeline **determinista**: el modelo no decide cuándo buscar ni cuándo derivar. Cada etapa crítica se resuelve en el backend, y las dos primeras cortan *antes* de que el mensaje llegue al LLM.

La tesis del proyecto

En un dominio de salud la seguridad tiene que ser un **mecanismo**, no una instrucción. Pedirle a un LLM que derive ante una emergencia falla el día que alucina o le inyectan contexto. Aquí nada crítico depende del modelo — y la telemetría de producción lo demuestra sin necesidad de un test.

En producción: <https://emociones-app-zxzgilzqfq-uc.a.run.app>

El guardarraíl se demuestra solo

La analítica de producción registra la latencia de cada turno. Al agrupar por tipo aparece algo que ningún test tenía que buscar:



Una emergencia se resuelve en **56 ms**; una consulta normal tarda **2.878 ms**. La diferencia —51 veces— es exactamente lo que el guardarraíl se salta: la recuperación y el modelo. Si la derivación fuera una instrucción en el prompt, ambos números serían iguales.

Datos reales de BigQuery. Es la comprobación empírica de NFR-02b: el mecanismo no puede ser evadido porque el mensaje nunca llega al LLM.

Por qué este dominio es distinto

El corpus es un diccionario de significados emocionales: 1.265 términos que asocian un síntoma físico con un conflicto emocional. Eso plantea tres riesgos que no existen en un chatbot cualquiera.

Riesgo 1 · Retrasar atención médica

Alguien con un síntoma grave podría quedarse con la interpretación emocional en vez de ir al médico. **Mitigación:** 156 términos marcados en 7 niveles de riesgo elevado reciben una plantilla que **encabeza con la derivación**, y 6 grupos de emergencia cortan antes incluso de recuperar.

Riesgo 2 · Lenguaje causal

Decir «tu resentimiento causó tu gastritis» es falso y dañino. El diccionario registra **asociaciones simbólicas**, nunca causas. **Mitigación:** hay un test que prohíbe patrones causales en las 8 plantillas y en las respuestas.

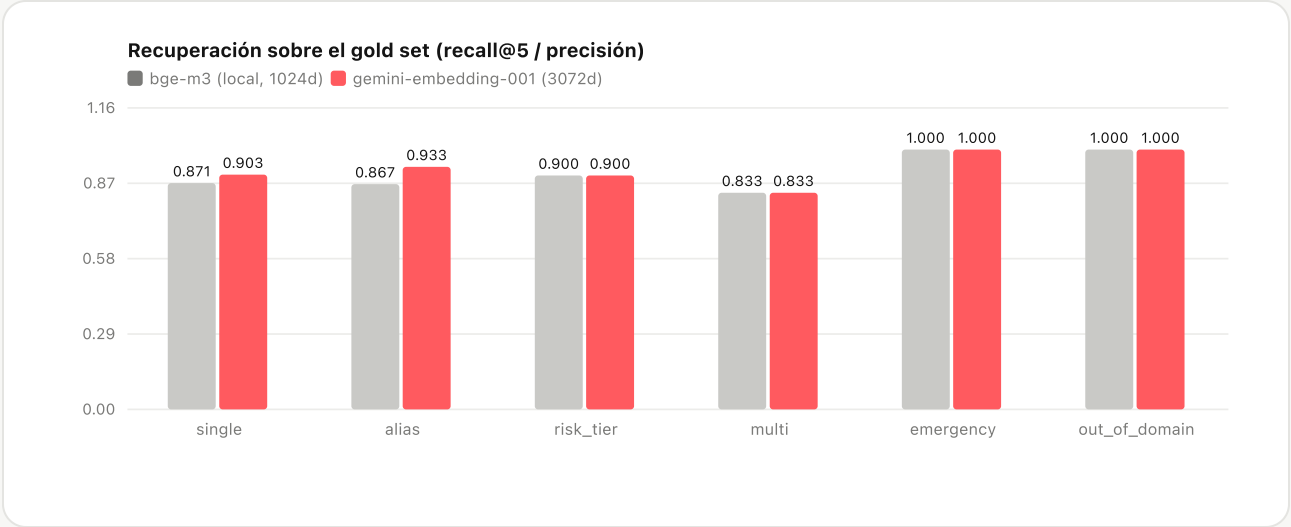
Riesgo 3 · Confabular fuera de cobertura

Un RAG que siempre devuelve k resultados siempre encuentra algo que decir, aunque la consulta no tenga nada que ver. **Mitigación:** un umbral de cobertura explícito y una respuesta «sin cobertura» honesta. Se priorizó **precisión fuera de dominio = 1.0** por encima del recall: en salud, inventar una interpretación es peor que admitir que no se sabe.

Cómo se decide qué es relevante

El corpus es un diccionario, y en un diccionario **el título es la señal fiable**: la prosa emocional de dos términos distintos es casi idéntica, así que el embedding denso por sí solo no discrimina. De ahí el diseño:

- **Cobertura** = existe match nominal normalizado de título/alias, con *light stemming* (plurales y deverbales) y stopwords. Ese match es el umbral.
- **Ranking** = fusión RRF de denso (FAISS, coseno) + BM25 con peso reforzado al título.
- **El denso solo desempata** dentro del mismo nivel de match.



TIPO	BGE-M3 (LOCAL)	VERTEX (PRODUCCIÓN)	META	ESTADO
single	27/31 = 0.871	28/31 = 0.903	≥0.85	✓
alias	13/15 = 0.867	14/15 = 0.933	≥0.90	✓
risk_tier	9/10 = 0.900	9/10 = 0.900	10/10	parcial
multi	5/6 = 0.833	5/6 = 0.833	6/6	parcial
emergency	5/5 = 1.000	5/5 = 1.000	5/5	✓
out_of_domain	13/13 = 1.000	13/13 = 1.000	1.0	✓

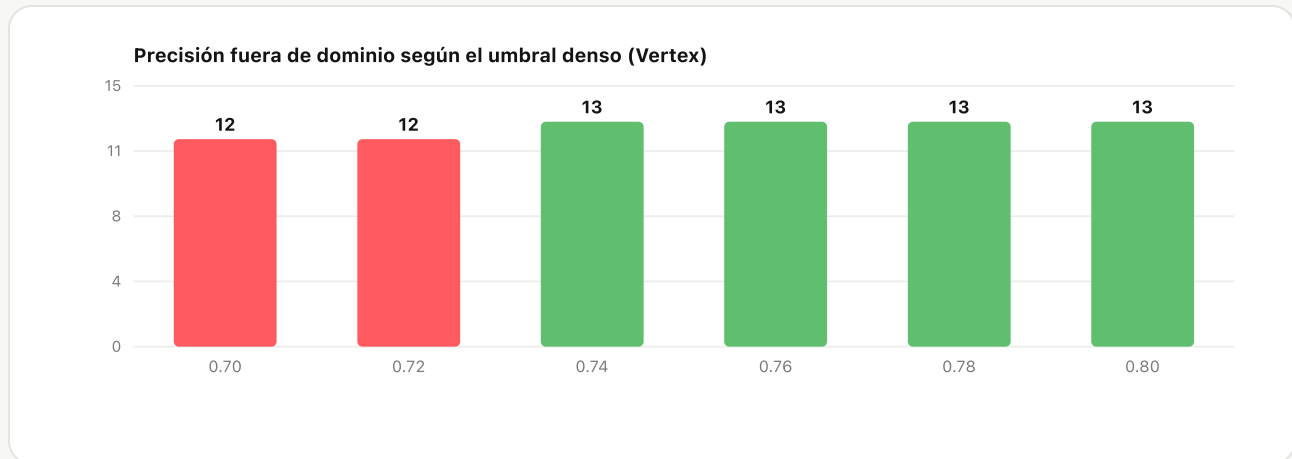
El gate duro de M2 se cumple por primera vez

alias pasa de 0.867 a **0.933** y supera el ≥0.90 que llevaba pendiente desde el segundo milestone. Los tres criterios del gate —single ≥0.85, alias ≥0.90 y precisión fuera de dominio 1.0— se cumplen ahora **simultáneamente**.

Un umbral correcto que se vuelve bug

La cobertura tiene un respaldo: si no hay match nominal pero el coseno denso es muy alto, se acepta. Ese umbral valía 0.70, fijado con datos... **para bge-m3**.

Al migrar a Vertex, la consulta «¿qué significa emocionalmente el cuerpo?» puntuaba **0.7385** sin ningún match nominal y se colaba como cobertura, **rompiendo la precisión fuera de dominio de 1.0**.

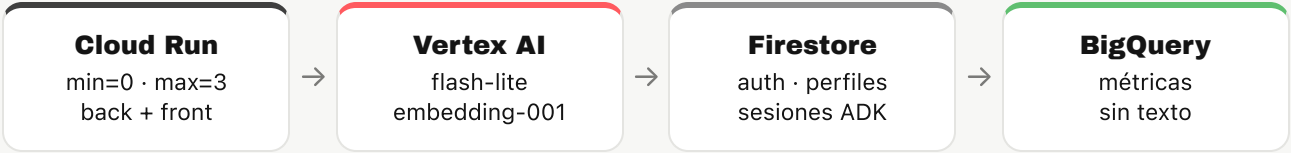


Se barrió el umbral sobre el gold set y se tomó el más bajo que conserva 13/13. Se fijó **0.75**, en medio de la meseta estable, y ahora es un **mapa por modelo**: un modelo sin calibrar recibe el valor más estricto.

La lección

Cada espacio de embeddings tiene su propia distribución de similitud. Un umbral heredado **no falla ruidosamente**: simplemente empieza a inventar interpretaciones para consultas que no cubre. En este dominio, ese silencio es el peor modo de fallo posible.

37 recursos, todo como código



Toda la infraestructura es Terraform: no se creó nada a mano en la consola. El índice FAISS (19 MB) va **horneado en la imagen**, así que **no hay servicio de vectores**. Y deliberadamente **no hay Cloud SQL ni VPC Connector**: son justo los componentes que corren 24/7 y no escalan a cero.

RECURSO	N	PARA QUÉ
google_project_service	9	APIs habilitadas
google_bigquery_table	4	consultas · recuperación · guardarraíles · eval
google_project_iam_member	4	los únicos roles a nivel proyecto
google_secret_manager_secret +versión +IAM	6	JWT y sal del hash analítico
google_firestore_index	2	historial por portador · eventos por sesión
google_cloud_run_v2_service +IAM	2	el servicio y su acceso público
google_bigquery_dataset +IAM	2	analítica
google_artifact_registry_repository +IAM	2	imagen, con limpieza automática
google_firestore_database	1	auth · perfiles · sesiones · auditoría
google_service_account	1	identidad del contenedor, sin claves
google_storage_bucket	1	estado remoto de Terraform
random_password	2	secretos generados, nunca escritos a mano

Vertex se autentica por service account, no por clave de API. Eso elimina un secreto entero del sistema: uno menos que rotar y que filtrar.

El presupuesto se hizo antes de crear nada

Los precios se consultaron y el consumo se **midió sobre el propio repo**: 3.400 tokens de entrada y 450 de salida por turno = **\$0.00052**, o \$0.52 por cada 1.000 turnos. Reindexar el corpus completo costó **\$0.09**, una vez.

La decisión que movía la aguja

Casi todo cabe en el free tier. Lo único que podía *no* escalar a cero era dónde viven auth y memoria:

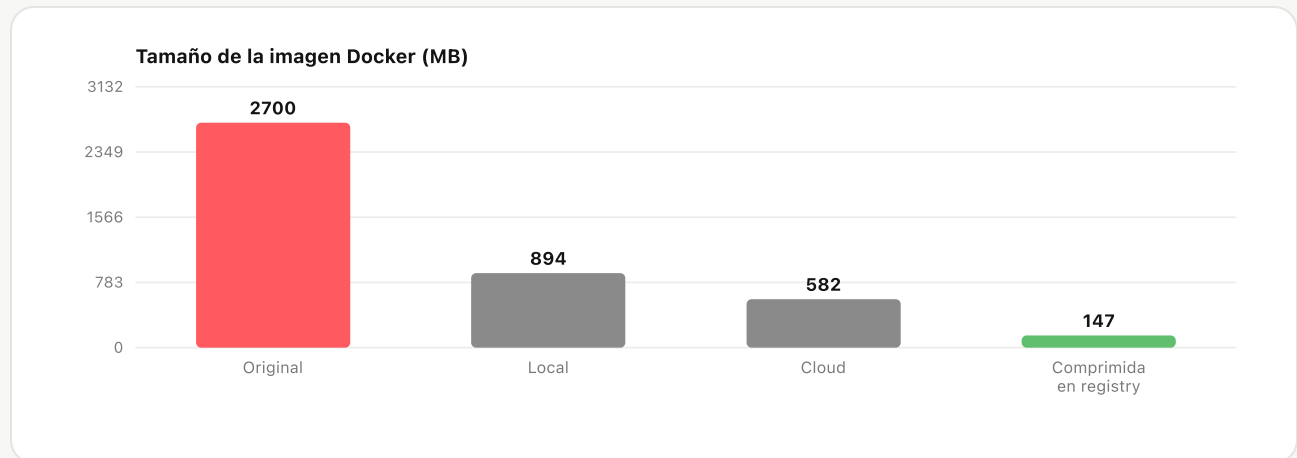
Costo mensual en reposo por alternativa (USD)



Se eligió **Firestore**. El precio no fue dinero sino código: ADK 2.8 no trae un session service de Firestore, así que hubo que implementarlo, y reescribir la persistencia tras un protocolo con dos backends intercambiables.

CONCEPTO	MENSUAL	NOTA
Cloud Run	\$0	6% del free tier con 5.000 turnos/mes
Firestore	\$0	dentro de 50k lecturas y 20k escrituras diarias
BigQuery	\$0	<1 GiB escaneado de 1 TiB gratis
Artifact Registry	\$0	147 MB comprimidos, bajo los 0,5 GB gratis
Secret Manager	\$0.12	2 versiones activas × \$0.06
Vertex (500 turnos)	\$0.26	medido, no estimado
Total con 500 turnos	≈ \$0.38	en reposo: \$0.12

Adelgazar la imagen: -95%

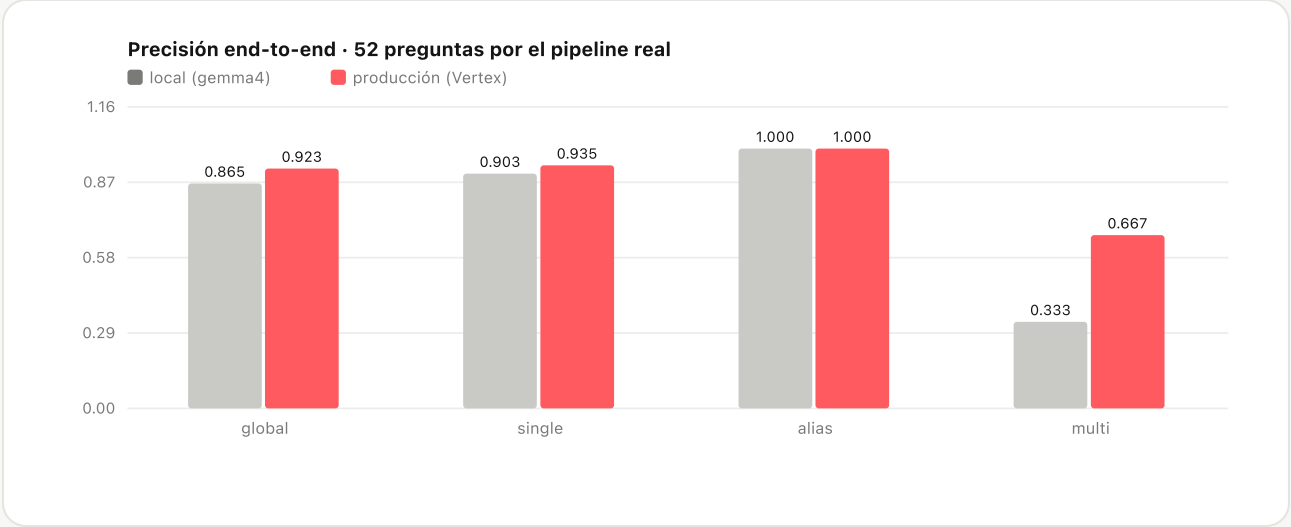


- Un `chown -R appuser /app` **después** de crear el `venv` duplicaba el árbol entero en una capa de **682 MB**. Ahora el build es multi-stage y el usuario se crea antes de copiar.
- `scipy` (71 MB) estaba en las dependencias y **no lo usaba ningún fichero**. `litellm` (92 MB, arrastra `botocore` y `openai`) solo hace falta en local.

El beneficio que más importa no es el costo sino el **arranque en frío**, que es el precio real de escalar a cero.

Qué tan bien funciona

52 preguntas derivadas del gold set, corridas por el **pipeline real del chat**. La verificación es **determinista**: acierta si cita el slug esperado o contiene el término. Sin LLM como juez. Métricas por bootstrap con seed fijo (42, 2.000 remuestreos).



MÉTRICA	N	LOCAL	PRODUCCIÓN	IC 95%
Precisión global	52	0.865	0.923	[0.846, 0.981]
Recall single	31	0.903	0.935	[0.839, 1.000]
Recall alias	15	1.000	1.000	[1.000, 1.000]
Recall multi-hop	6	0.333	0.667	[0.333, 1.000]
Latencia media	52	8,5 s	2,1 s	—

Honestidad sobre estos números

1. **La precisión fuera de dominio de 1.0 se compró a costa de recall.** El precio son los fallos de sinonimia coloquial (*panza*→*estómago*, *dormir*→*insomnio*) ausentes del fichero de alias. Fue una decisión, no un accidente.
2. **Multi-hop sigue siendo el punto débil** (0.667). Citar *todos* los síntomas en una síntesis cohesiva es lo más difícil, y arrastra el mismo hueco.
3. **El LLM no es determinista.** Entre corridas locales se observó 0.865–0.885. Lo reproducible es el *procedimiento*, no la cifra exacta.
4. **Buena parte del salto de multi-hop no es mérito del modelo**, sino de un bug corregido —ver §11.

Tests que pueden fallar

260 tests por área



253 passed + 7 skipped. Los saltados necesitan un Postgres alcanzable desde el host y cubren aislamiento, persistencia y borrado en el backend local.

173 tests de seguridad médica

Los **156 términos** de riesgo elevado, cada uno con su plantilla de derivación. Cero patrones causales en las 8 plantillas. Y los 6 grupos de emergencia con `retrieval.search` monkeypatcheado para **fallar si llega a llamarse**: eso prueba que el corte ocurre *antes* de recuperar.

17 aserciones de infraestructura

Sobre el terraform plan **resuelto**, no sobre el texto de los `.tf`: un comentario no engaña a un plan. Roles prohibidos, alcance por recurso, `allUsers` solo en el invoker, bucket de estado privado, escala a cero, y que BigQuery no admita ningún campo capaz de llevar texto de chats.

Validadas por mutación, no solo verdes

Un test que no puede fallar no prueba nada. Al inyectar en el plan `roles/owner`, un campo mensaje en BigQuery y `min_instances = 1`, **fallan 5 aserciones**; al restaurar el plan, vuelven las 17 en verde.

scripts/secrets_audit.sh

Auditoría de secretos – 8/8

- ✓ .env fuera de git
- ✓ sin literales de secreto en ficheros versionados
- ✓ la imagen no contiene .env ni credenciales
- ✓ sin literales de secreto en el código Python
- ✓ sin .tfstate ni .tfvars versionados
- ✓ sin secretos en claro en los .tf
- ✓ no se crean claves de service account
- ✓ el plan no expone valores sensibles

Pruebas de uso en producción — 16/16

Ejecutadas contra el servicio vivo con `scripts/prod_smoke.py`. **Cada caso lleva su asección:** si una falla, la corrida falla. La evidencia no puede decir «OK» sin haberlo comprobado.

CASO	FR/NFR	DETALLE OBSERVADO
health	—	✓ HTTP 200
frontend servido	FR-18	✓ HTTP 200
registro	FR-16	✓ HTTP 200, token emitido: True
login con clave errónea rechazado	FR-16	✓ HTTP 401
perfil sin token rechazado	FR-17	✓ HTTP 401
consulta con fuentes citadas	FR-03/FR-09	✓ kind=respuesta, 3 fuentes, 3.1s
multi-hop	FR-04	✓ 4 fuentes, 2.4s
emergencia deriva sin interpretar	FR-06	✓ kind=emergency, 0 fuentes, 585 ms
emergencia más rápida que una consulta normal	NFR-02b	✓ 585 ms vs 3079 ms
prompt injection bloqueado	NFR-01	✓ HTTP 403
fuera de dominio sin confabular	FR-09b	✓ kind=sin_cobertura
historial persistido	FR-14	✓ 2 consultas
sesiones ADK persistidas	FR-12	✓ 1 sesión(es), 8 eventos
responde desde el historial	FR-13	✓ Tu última consulta fue el 7 de septiembre de 2026, donde preguntaste s
usuario B no ve datos de A	FR-17	✓ 0 consultas y 0 sesiones ajenas
borrado del historial	FR-14b	✓ historial vacío tras DELETE

La interfaz, capturada sobre el sistema real

Chrome headless sobre la UI desplegada. Cada escenario verifica su aserción en el DOM: si los chips de fuentes no se ven *en el viewport*, si falta la clase de emergencia o si un GIF no tiene transiciones reales, la corrida falla en vez de producir una imagen vacía.

Acceso y alta de usuario (FR-16/FR-18).

Respuesta con chips de fuentes y disclaimer visible (FR-19/FR-20).

Emergencia: derivación resaltada y sin fuentes, porque no llegó a recuperar (FR-06).

Prompt injection bloqueado con 403 (NFR-01).

```
PERFIL DEL PORTADOR (GET /profile)
{
  "user_id": "evidencia-83308086@demo.co",
  "consultations": [
    {
      "ts": "2026-09-07T05:07:06.511013+00:00",
      "term": [
        "garganta-dolores-de",
        "garganta-en-general",
        "garganta-apretada",
        "garganta-laringitis",
        "garganta-faringitis"
      ],
      "symptom": [
        "dolor de garganta"
      ]
    }
  ]
}
```

Historial persistido del portador en Firestore (FR-14).

```
SESIONES ADK PERSISTIDAS (GET /sessions)
{
  "sessions": [
    {
      "session_id": "s-hixqss9y",
      "user_id": "evidencia-83308086@demo.co",
      "app_name": "ah_emociones",
      "events": 6,
      "state": {}
    }
  ]
}
```

Sesiones ADK persistidas (FR-12).

Analítica en BigQuery

El dataset se alimenta desde el propio turno de chat. La regla dura: **métricas y metadatos, nunca el texto de las conversaciones**; el usuario es un hash con sal, no su correo.

TABLA	PARA QUÉ SIRVE
consultas	términos consultados, nivel de riesgo, latencia por turno
recuperacion	qué consultas reales quedan sin cobertura — convierte la deuda de sinonimia en algo medible
guardarrailes	cuántas veces dispara cada guardarraíl, y cuál
eval	serie histórica de las corridas: si el recall se degrada entre versiones, se ve

Lo que solo apareció al desplegar

El despliegue real destapó cuatro cosas que ninguna suite local había detectado. Vale la pena contarlas: son el resultado más útil del proyecto.

1 · Una fuga de privacidad en producción

Al inspeccionar las filas reales de BigQuery, el campo `symptom_slug` contenía **el mensaje literal del usuario**. Cuando la extracción de síntomas no devuelve nada, el pipeline busca con el mensaje entero y esa lista se emitía tal cual.

El test que cubría esto no lo detectó: usaba un caso de emergencia, donde la lista de síntomas va vacía. El camino con fuga era el de «sin cobertura». Corregido, datos purgados recreando la tabla, y test de regresión sobre el camino correcto — validado por mutación.

2 · El agente llevaba degradado sin avisar

`gemini-2.5-flash-lite` devuelve el JSON de extracción **envuelto en un bloque markdown**; `gemma4` lo devuelve pelado. `json.loads` fallaba y el sistema caía al fallback **en todos los turnos**, buscando con el mensaje entero en vez de con cada síntoma por separado.

```
diagnóstico

$ raw = _complete(_EXTRACT_PROMPT, "se me cae el pelo y me salen granos")
RAW: ```json\n[\n  "caída del pelo",\n  "granos"\n]\n```
# json.loads() falla -> fallback -> multi-hop degradado, sin excepción
```

Corregirlo subió `multi` de 0.500 a **0.667** y el `global` a **0.923**. Buena parte de la mejora atribuible «al modelo» era en realidad un bug.

3 · Cloud Run reserva el prefijo `ah-`

El servicio `ah-emociones-app` se rechaza con un 400 en pleno `apply`. Se separó el nombre del servicio con una `validation` en Terraform, para que el próximo error salga en el `plan` y no a mitad del despliegue.

4 · Código escrito, testeado... y nunca llamado

El emisor de analítica existía y tenía tests verdes, pero **nadie lo invocaba**: las tablas quedaron vacías tras el primer despliegue. Probar un módulo aisladamente no detecta que esté desconectado. Ahora hay tests que corren el **pipeline completo** y exigen que emita.

Deuda abierta y cómo reproducirlo

Lo que queda abierto, declarado

- **Sinonimia coloquial** ausente del fichero de alias, congelado desde el inicio. Cerrarla pide un fichero de sinónimos aparte, no tocar el congelado. Es la causa raíz tanto de multi 0.667 como de risk_tier 9/10.
- **La alerta de presupuesto hay que crearla a mano.** La API de Billing Budgets devuelve 400 en esta cuenta de facturación; se descartó que fuera la configuración comprobando que falla igual un budget mínimo creado con gccloud sin filtro. Es la red de seguridad contra un gasto inesperado de Vertex, lo único que escala con el uso.
- **Los tests del backend de nube usan un doble en memoria**, no el emulador de Firestore. El despliegue real fue la primera vez que el código habló con Firestore de verdad — y ahí aparecieron dos de los cuatro fallos del postmortem.

Reproducir todo

● ● ● desde la raíz del repo

```
scripts/test.sh           # 253 passed + 7 skipped
scripts/evidence.sh       # eval e2e → capturas → GIFs → reportes
scripts/infra_audit.sh    # fmt + validate + plan real + 17 aserciones
scripts/secrets_audit.sh  # 8/8
scripts/prod_smoke.py     # pruebas contra el servicio vivo
scripts/final_report.py   # este documento
```

Qué se lleva el proyecto

Que en un dominio con consecuencias, **la seguridad tiene que ser verificable, no prometida**. Los guardarraíles se resuelven en el backend y se prueban con tests que pueden fallar; las capturas se autocomprueban; las aserciones de infraestructura se validan por mutación. Y aun así, el despliegue real encontró cuatro cosas que nada de eso había visto — incluida una fuga de datos de salud. Esa es la parte honesta del resultado.